科研小助手

面向科研场景的 AI 文献理解与写作 Agent

技术文档

第九届传智杯大赛 · AI 大模型创新应用挑战赛

参赛作品技术文档

2026 年 10 月

一、作品概述

1.1 一句话定位

科研小助手是一个本地部署的文献理解与写作 Agent。用户把手里的 PDF 论文拖进来，系统完成细粒度解析、混合检索问答、中英双向翻译、引用关系图谱分析，并在写作环节提供受约束的框架生成、架构图绘制、数据可视化与中译英。全部功能围绕「让研究者少花时间在找和读，多花时间在想和写」这一个目标展开。

1.2 与赛题要求的对应

赛题将考察点拆为六大功能模块。下表给出模块、实现位置与实测证据的对应关系，所有数字均为本机真实库（39 篇 PDF）上的运行结果。

官方模块	实现文件	实测证据
1.1.1 论文去重	app/papers.py	文本相似度阈值 0.80；v1/v7 实判 0.873
1.1.2 细粒度解析	app/structure.py	39 篇 / 961 章节 / 282 表 / 287 图
1.2 知识问答	app/agent.py + app/rag/	逐句带页码引用；无证据时明确拒答
1.3 阅读与翻译	app/reader.py + app/translate.py	章节树、双向跳转、划词/悬停翻译
1.4 下载与推荐	app/online.py + app/recommend.py	arXiv + GitHub 两路检索，反馈调优
1.5 分析与生成	app/graph.py	36 节点 / 189 引用边
1.6 写作助手	app/writing.py 等 4 个模块	候选集约束生成、三渲染器、CSV 出图、中译英

1.3 工程规模

维度	数值	统计口径
后端代码	14,320 行	app/ 下全部 .py
前端页面	24,938 行	web/ 下 .html/.css/.js，不含 vendor
测试代码	6,145 行	tests/ 下全部 .py

脚本	5,645 行	scripts/ 下全部 .py
REST 接口	44 个	app/main.py 路由装饰器
测试用例	pytest 收集 444 个，全部通过	pytest --co 实测
自动化检查	ruff 全绿 + 依赖自检 + CI 三步	见 8.3

二、系统架构

2.1 分层结构

系统采用「单进程 Web 服务 + 本地数据目录」的形态，不做分布式拆分。理由很实际：这是一份需要评委在浏览器里点得动的作品，多部署一个中间件只会增加演示失败的概率。

```
浏览器（原生 HTML/JS，无构建步骤）
        │  fetch / SSE
        ▼
FastAPI 应用 app/main.py  ── 44 个 REST 接口
        │
        ├─ app/agent.py    编排：规划→检索→抽取→校验→写作
        ├─ app/rag/        三路召回 + RRF 融合 + 交叉编码器重排
        ├─ app/graph.py    引用图谱与图谱挖掘算法
        ├─ app/writing.py  候选集约束生成
        ├─ app/chart.py / diagram.py / zh2en.py  写作三件套
        └─ app/structure.py / ocr.py / layout.py  解析层
        │
        ▼
data/   papers(原始 PDF) · chroma(向量) · meta(解析与索引) · models(本地模型)
```

2.2 数据流：一次问答发生了什么

- 前端 POST /chat/stream，服务端返回 SSE 事件流，逐条推送角色进度与证据。
- Planner 判定是单篇聚焦、多篇对比还是全库检索，并生成检索词。
- Retriever 走三路召回（稠密 / 哈希 / BM25），RRF 融合后取前 15 个候选。
- 交叉编码器对候选重排，保留相关性最高的若干片段作为证据。
- Extractor 在需要时补充结构化材料（论文卡片、表格行、章节原文）。
- Verifier 统计证据条数、覆盖论文数、可定位页码数，缺页码会标红。
- Writer 只依据上述证据撰写，每句标注 [来源: 文件 第 N 页]，证据不足则明确说明。

2.3 数据组织

路径	内容	是否入库

data/papers/	原始 PDF，只读，任何功能都不修改	否（体积大）
data/chroma/	向量索引，可用脚本重建	否
data/meta/structures.json	细粒度解析结果	是
data/meta/citation-graph.json	引用图谱，可用脚本重建	否
data/models/	本地 ONNX 模型（重排、Embedding）	否
data/catalog.json	论文元数据清单	是

「是否入库」这一列是刻意设计的结果：索引与图谱都能由脚本从原始 PDF 重建，放进仓库只会让克隆变慢且容易产生二进制冲突。原始 PDF 不进仓库则有另一个理由——公开仓库不应分发他人版权论文。

2.4 工具调用链：Function Calling 的接入方式

赛题评分细则把「Function Calling 与工具集成」单列 6 分，并要求工具调用可观测。本项目的接入点集中在 app/agent.py，调用方式是标准的 OpenAI tools 协议，不额外封装一套自造协议。

```
# app/agent.py:1497  按工具说明书请求模型
get_client().chat.completions.create(
    ..., tools=tool_schemas(), tool_choice="auto", temperature=0.3,
)

# app/agent.py:52  一轮问答内最多允许多少次工具往返
MAX_ROUNDS = 5

# app/agent.py:1441  按名字分发执行
async def _execute_tool(name, arguments): ...

# app/agent.py:1572-1573  结果以 role=tool + tool_call_id 回填上下文
{"role": "tool", "tool_call_id": call.id, "content": result}
```

工具本身是插件式的：继承 app/tools/base.py 的 BaseTool，实现 to_openai_schema()，再到 app/tools/registry.py 的 _ALL_TOOLS 里登记一行即可被模型看到。当前实际注册 1 个工具（计算器），注册表里另有两个位（课表、笔记）以注释形式预留。这里刻意只登记真实可用的工具——把未实现的工具塞进 schema，模型会去调用一个空壳，比没有工具更糟。

计算工具不走 eval，而是走 AST 白名单：只允许 Add / Sub / Mult / Div / Pow / Mod / USub 七类节点，其他一律拒绝。理由很直接——eval 一个模型生成的字符串，等于把进程的执行权交给上游模型。

- 调用过程通过 SSE 的 tool_call / tool_result 两个事件实时推给前端，用户能看见模型调了什么、拿到什么。
- 工具结果只作为下一轮的上下文，不直接进入最终答案，仍需走 Writer 的证据标注流程。
- 一轮问答最多 5 次工具往返，超过就强制收尾并在时间线上标红，避免模型陷入死循环。

三、核心技术实现

赛题评分中「AI 技术深度」占 35%，原文列举了文档智能解析、高级 RAG 架构、多智能体协同三项。本章按这三项展开，并补充本地模型部署情况。

3.1 文档智能解析

3.1.1 四层降级策略

论文 PDF 的复杂度差异很大：有原生文本层、有双栏排版、有纯扫描件、有 541 页的书籍。系统按四层依次尝试，任一层失败就降级到下一层。

层级	触发条件	实现	代价
1 原生文本层	PDF 有可提取文本	pymupdf 按块提取	最快（毫秒级）
2 版式感知	检测到双栏	app/layout.py 按栏切分再排序	加几十毫秒
3 表格专项	页面含表格结构	pdfplumber 文本策略	较慢但准确
4 OCR 兜底	文本层为空（扫描件）	RapidOCR + ONNX	秒级，按页累计

实测覆盖：39 篇论文全部解析成功，其中包含扫描件 1706.03762v1_img.pdf 与 541 页的 Understanding Deep Learning。

3.1.2 结构化抽取的难点

真正花时间的地方不是提取文字，而是从版面里可靠地认出「哪一行是作者」。论文的作者区排版千奇百怪，实测踩到并被修复的五类失败模式：

- LaTeX 排版里作者之间没有逗号，只能靠机构标记切分。
- 名字后面紧跟脚注数字，如「Ashish Vaswani1」，不清掉数字就认不出名字边界。
- 姓名与上标编号粘连，如「Niki Parmar2,3Google Research」。
- 整行是机构署名而非人名，如「Google Brain」，会被误当成作者。
- 分组标题混在作者里，如 BLOOM 的「BigScience Workshop」「Major Contributors」不是人名；但「Llama Team, AI @ Meta」是机构作者，不能误删。

修复后作者抽取覆盖 38/39 篇，全部使用真实人名而非占位符。这条链路有独立回归测试 tests/test_authors.py（215 行）。

3.2 高级 RAG 架构

3.2.1 三路召回与融合

单路检索在学术文本上都有明显短板：稠密向量擅长语义但会漏专有名词，BM25 擅长精确匹配但不懂同义词，哈希向量作为轻量兜底。因此三路并行召回再融合。

通道	实现	权重	擅长
稠密	chromadb ONNXMiniLM_L6_V2（384 维）	2.0	语义相近但用词不同
哈希	自建 1024 维特征哈希	0.5	无模型时的兜底
BM25	自建（k1=1.2, b=0.6）	1.0	术语、公式号、人名精确命中

融合用 Reciprocal Rank Fusion，k 取 60：

```
fused[cid] += weight / (RRF_K + rank + 1)      # RRF_K 默认 60
```

权重可由环境变量 `RAG_W_DENSE / RAG_W_HASH / RAG_W_BM25` 覆盖

选 RRF 而不是加权分数相加的理由：三路通道的原始分数量纲完全不同（余弦相似度 0~1、BM25 无上界、哈希内积可正可负），直接相加等于让量纲大的通道主导结果。RRF 只用名次，天然规避了这个问题。

3.2.2 交叉编码器重排

召回阶段是「双塔」结构——问题和文档分别编码，速度快但精度有限。重排阶段换成「交叉编码器」：把问题与文档拼在一起送入模型，让它直接看两者的交互，精度高但慢，所以只用于重排前 15 个候选（RAG_RERANK_POOL）。

项	取值
模型	ms-marco-MiniLM-L-6-v2（ONNX 量化版，约 22 MB）
输入截断	512 token（BERT 类模型上限）

输出	logits（可正可负，只在批内可比）
缺模型时	抛 RerankUnavailable，调用方回退到 RRF 排序，检索不中断

关于输出是 logits 这件事，前端一度把它当成「融合分」显示，导致出现负数分数。已修正为按 score_type 区分显示：RRF 分叫「融合分」（恒正），重排分叫「重排分」（可为负，且只在同一问题的候选间可比）。

3.2.3 抗幻觉：候选集约束生成

这是本项目在写作环节最核心的一个设计决策，解决的问题是：大模型写综述时会凭空编造参考文献。常见做法是事后校验——生成完再检查引用是否真实，不合格就重写。本项目的做法是让编造在结构上不可能发生。

① 用 idea 检索真实文献，得到候选池，每篇分配 key：C1，C2，... Cn

② 候选池连同 key 一起塞进 prompt，明确要求：只能引用池内的 key

③ 模型只写标记 [C3]，不允许写出完整条目

④ 参考文献列表由代码从候选池取原始元数据生成，模型碰不到

⑤ 生成后校验：出现池外 key 即判失败

效果：实测生成 12 篇候选、正文引用 12 个 key，悬空 key 为 0。池子为空时系统不会硬生成，而是直接返回「没有检索到任何真实文献，为避免编造引用，未生成框架」。

同样的思路贯穿问答环节：证据不足时不编造，而是明确说明资料不足。实测提问「Attention 的核心机制是什么？」时，由于检索到的片段没有给出形式化定义，回答主动写明「检索片段只给出零散描述，没有 attention 的形式化定义……资料不足」。

3.3 多智能体协同

3.3.1 为什么不是四个模型互相聊天

调研阶段读了 MA-RAG、paper-qa、med-paper-assistant 等实现，主流做法是多个 Agent 通过共享状态或消息队列协作。但直接照搬会遇到一个硬约束：四角色互调意味着单次提问要多调 3~4 次大模型，现有单问耗时 6~20 秒，加倍后会超出演示可接受范围。

因此本项目采取「显式命名 + 可核验产物」的路线：把原本隐式串在一起的处理阶段显式建模成四个角色，每个角色留下可核验的产物与耗时，但不为了「像多智能体」而增加模型调用次数。

角色	职责	产物
Planner 规划	判定单篇/多篇/全库，生成检索词	计划类型 + 检索词
Retriever 检索	三路召回 + 融合 + 重排	召回片段数、覆盖论文数
Extractor 抽取	补充卡片/表格行/章节原文	结构化材料条数
Verifier 校验	统计证据与页码可定位性	证据数、页码完整性
Writer 写作	依据证据撰写并标注来源	字数、轮次

实测输出示例：

```
Status: 证据=4 · 覆盖论文=2 · 页码=2,11
Planner   0ms    全库检索 · 检索词「core mechanism of attention」
Retriever 2928ms 召回 4 段原文，覆盖 2 篇论文
Extractor 163ms  本次问题不需要结构化加工，直接用检索片段
Verifier  0ms    证据 4 条，其中 4 条可定位到具体页码
Writer    3337ms 生成答案（912 字，第 1 轮定稿）
```

这条时间线直接渲染在前端回答上方，用户可以看清系统做了什么、哪一步慢、证据够不够。Verifier 判定为失败时（证据缺页码、或完全没有证据）该行标红。

3.4 本地模型部署

评分标准原文写明「本地模型部署加分」。本项目在三个环节使用本地模型，均为 ONNX 运行时，不依赖网络：

用途	模型	体积	离线
文本嵌入	ONNXMiniLM_L6_V2（374 维）	约 80 MB	是
交叉编码器重排	ms-marco-MiniLM-L-6-v2 量化	22 MB	是
扫描件 OCR	RapidOCR（PP-OCR ONNX）	约 15 MB	是

只有最终撰写环节调用 DeepSeek API，且该调用可替换为任意 OpenAI 兼容端点（配置项 llm_base_url），演示环境可切换本地推理服务。

四、引用关系图谱

引用图谱是本项目在「学术算法与图谱挖掘创新」上投入最多的部分。难点不在画图，而在数据：参考文献条目是 PDF 里最难解析的文本之一。

4.1 从 PDF 到图谱的三道关口

4.1.1 第一关：把参考文献条目切分干净

初版实现里约 3300 条参考文献未能抽出标题，人工抽查发现典型形态：标题被截断成作者姓名的碎片、条目跨栏被切断、跨页条目断成两半。处理策略是「多信号切分」——综合标点、年份位置、作者块长度、缩进与续行标记，而不是依赖单一正则。

4.1.2 第二关：把同一篇论文的多个副本合并

库内存在同一篇论文的不同版本与扫描副本。若直接建图，Attention Is All You Need 会被拆成三份，它的被引数和 PageRank 都会失真。系统按「标题 + arXiv 号 + 元数据」交叉核验后合并为单一逻辑节点，实测 1 个节点被合并，其中 arxiv:1706.03762（4 份）。合并不删除任何 PDF 文件。

4.1.3 第三关：三层匹配，宁缺毋滥

层	匹配依据	置信度	出边
1	arXiv 号（归一化前缀、空格、版本号）	最高	是
2	完整规范化标题（大小写/空白/标点/Unicode 全归一）	高	是
3	标题近似（相似度 ≥ 0.92）	中	是
—	相似度落在 0.75~0.92 之间	低	否，记入待人工核验

这条「相似度落在中间区间就不出边」的规则是刻意设计的。标题近似匹配最大的风险是误连不同论文，产生虚假的学术关系。宁可少几条边、把可疑的列出来给人看，也不要让图谱好看但结论错。当前图规模：36 个逻辑节点、189 条引用边。

4.2 图谱挖掘指标

三个指标必须分开统计，否则会得出错误结论，这也是数据里最容易踩的坑：

指标	含义	为什么要区分
引用边数量	入边总数	同一篇论文可能被多篇多次引用
被引论文数量	有多少篇不同的论文引用了它	反映传播广度
不同引用论文数量	去重后的引用方个数	与上项区分，避免重复计数

PageRank 在去重后的逻辑节点上计算，不经过副本重复加权。此外还实现了中介中心性（找跨社区桥接论文）与 Louvain 社区发现。值得注意的是这些算法全部由项目自己用标准库实现，刻意不依赖 networkx / scipy——依赖越少，他人复现时踩坑越少。

4.3 三种图谱挖掘产物

- 基石节点：PageRank 高但入度不高，说明被少量关键工作引用、影响超出直接连接范围。
- 自动综述：基于图结构与元数据生成研究现状概览，不依赖大模型臆测。
- 前沿探索：三路信号——新近且被引、跨社区桥接、未被接入引用网。

五、写作助手

5.1 框架生成

见 3.2.3 的候选集约束生成。补充一个细节：本地论文的作者名单原本读不到，因为论文清单里只有作者数的计数字段，没有名单本身。修复方式是回到细粒度解析结果里按文件名索引作者，使参考文献能列出真实作者。超过 6 位作者按 IEEE 惯例截断为前 6 位加 et al.，并修掉了 et al. 与句号重叠成双句点的问题。

5.2 架构图渲染

提供三种渲染器，覆盖不同投稿场景：

渲染器	产物	适用场景	实现要点
Mermaid	PNG	快速预览、文档内嵌	语法接近自然语言，模型不易写错
Graphviz	PNG	黑白方框，最像论文图	注入 CJK 字体，否则中文全是方框
TikZ	PNG	LaTeX 投稿，原生矢量	编译失败只回喂错误重试一次

工程里有三个真实的坑值得记录：Windows 上 shutil.which 会优先命中 npm 的无扩展名脚本导致「不是有效的 Win32 应用程序」，改为按 .cmd/.exe 顺序手动查找；mermaid-cli 12 移除了 -w 参数改为 -s；Graphviz 默认字体不含中文字形，需在生成的代码里注入 fontname。

5.3 数据可视化

CSV 上传后自动做统计画像、选图型、出交互预览与定稿 PNG。三个决策值得说明：

- 双路输出：ECharts 供浏览器交互预览，matplotlib 供投稿导出。服务的是论文流程的不同环节。
- 对数轴阈值取 100 倍：参数量从 0.34B 到 540B 跨度 1600 倍，线性轴上小模型全挤在原点；而准确率 86~93 不跨数量级，用对数轴反而误导。
- 雷达图逐列归一化：各维量纲不同，不归一化会让大数值维度吃掉整张图。

Caption 生成有一条纪律：只给模型统计摘要，不给图。否则模型会描述出图上并不存在的趋势。实测生成的 caption 里所有数字都来自统计摘要。

5.4 中译英

与阅读页的英译中方向相反，提示词完全另写。目标是译文能直接塞进 IEEE/ACM 模板，因此硬性要求：只输出译文、正式学术语域、术语一致、长句拆短、数字单位与引用标记原样保留、时态遵循学术惯例、不增不减。支持术语表（每行「中文=English」），分块翻译时把上一块末尾作为上下文，保持术语连贯。

六、工程质量

6.1 测试

pytest 实测收集 444 个用例，覆盖解析、检索、图谱、写作、接口与边界条件。一条重要的测试哲学：只测确定性逻辑，不测大模型输出质量。

例如图谱测试里，若断言「某条文献应被匹配到某篇论文」，正则或阈值任何一侧微调都会让测试红掉，但功能可能更好。因此测试聚焦在可判定的部分：引用校验、去重逻辑、分块边界、选图规则、字体注入是否发生。而「两篇标题很像时该不该连边」这种规则断言，则必须测——它是正确性问题。

写测试时还踩到一个真实教训：构造「相似但不同」的用例必须确认相似度真的低于阈值。最初用「A Survey of Large Language Models」对「Survey of Large Language Models」做拒连用例，实测相似度 0.9688，高于默认阈值 0.92，系统本来就会连边——用例本身就是错的。

6.2 依赖治理

本项目遇到的最大的一个工程问题：依赖清单与代码脱节。实际 import 的 8 个包（pymupdf、pdfplumber、matplotlib、pandas、numpy、tokenizers、rapidocr、chromadb）一个都不在清单里，意味着任何人克隆后执行 pip install -r requirements.txt 都起不来服务。

修复的同时加了自检脚本 scripts/check_deps.py：用 AST 扫描 app/ 的真实 import，与清单比对，缺失即退出码 1，已挂进 CI。这样同类问题不会再悄悄发生。同时删掉了 6 个从未被 import 的包（redis、sqlalchemy、pymysql、aiohttp、sse-starlette、tiktoken），减少安装体积与攻击面。

6.3 容器化

Dockerfile 基于 python:3.11-slim（选 Debian 而非 alpine：chromadb、onnxruntime、matplotlib 依赖 glibc 与预编译 wheel，alpine 的 musl 会让 pip 现场编译）。额外装了 graphviz、libgomp1 与 fonts-noto-cjk——最后一个是为了让容器内导出的图表中文不变方框。.dockerignore 把构建上下文从 2.1 GB 压到 27 MB。

6.4 安全

- API 密钥只通过环境变量读取，.env 已加入 .gitignore 并在提交前逐项人工核查。

- 原始 PDF 只读，任何功能都不修改用户文件。
- 索引与图谱均可从原始 PDF 重建，损坏不影响数据。

6.5 设置中心：改配置不用登服务器

部署到云服务器之后，最烦的一件事是：想换个模型、把检索条数从 15 调到 20，得 SSH 上去改 .env 再 systemctl restart。设置中心把这件事搬到浏览器里。页面独立为 web/settings.html，从主界面齿轮进入，分五个模块：模型设置、RAG 检索设置、定时刷新、知识库管理、关于项目。

可变项共 13 个，全部登记在 app/settings_api.py 的一张表里，每一项声明 key、类型、默认值、取值范围和影响哪个运行时属性。这样新增一个可配项只需加一行，不用改前端也不用改路由。

设计点	做法	为什么这么做
原子写回 .env	按行替换，写完临时文件再 os.replace	直接整体重写会丢掉注释与其他键；中途失败会留下半个文件
双写 os.environ	改完同时更新进程环境变量	环境变量优先级高于 .env，只改文件不重启不生效
保存失败整体回滚	任一项校验或写盘失败，已改的项全部还原	避免留下「一半新配置一半旧配置」的状态
密钥脱敏	读出来显示 sk-****abcd，写回时识别掩码不覆盖真实值	既能让用户确认填的是哪把 key，又不在页面上暴露明文
环境变量覆盖提示	检测到启动环境有同名变量时在页顶告警	否则用户改了以后重启又变回去，会以为是 bug
单项恢复默认	每项右侧一个恢复按钮，只还原该项	不需要为了改一项而重置整份配置

6.6 一个静默失效的 bug 及其修复

设置中心第一版有个坑，值得单独写出来，因为它属于「测试全绿但功能没生效」那类。RAG 重排开关的配置键是 RAG_RERANK，代码按既定规则推导出运行时属性名 _RERANK，但 app/rag/pipeline.py 里真实存在的变量叫 _RERANK_ENABLED。推导失败后被 try/except 吞掉，页面上保存成功、返回 200，实际什么都没变。

修法有两层：一是给 Setting 增加一个显式 attr 字段，允许直接指定属性名，不再靠字符串拼接猜；二是 _apply_runtime 里对每个属性做 hasattr 检查，找不到就抛异常而不是静默跳过。第一层解决这一处，第二层保证同类问题以后会以报错的形式暴露，而不是以「看起来成功了」的形式隐藏。

6.7 知识库健康检查与幽灵清理

向量库有一个容易积累的脏数据问题：源 PDF 被删了，但索引里的切块还在。这些「幽灵」不会被检索时发现异常，只会让检索结果里偶尔出现一篇根本不在库里的论文。实测本机曾积累 4 个幽灵 source、共 851 个切块。

app/kb_health.py 提供两项能力：健康检查对比磁盘上的 PDF 与索引里的 source，列出差异；幽灵清理复用 pipeline.remove_source 逐个删干净。清理前必须在页面二次确认，因为这个操作不可撤销。修复这一层时还发现 docs_dense 集合的删除失败被 except Exception: pass 静默吞掉，会留下普通集合与稠密集合不一致的状态——已改为记录 warning 并抛 RuntimeError。

6.8 前端设计系统：把七个页面收敛成一份

项目最初是七个页面各写各的样式，同一个灰色有四种写法，改一次主题色要改七处，而且改漏了没人发现。这一阶段的改造把这层收敛掉了。

项	改造前	改造后
颜色与间距来源	各页面硬编码 hex / rgba	web/design-system.css 一份，172 个 CSS 变量
主题切换	改七处，容易漏	改主题层一组变量即可
图表配色	ECharts 里写死色值	web/chart-colors.js 读 CSS 变量，33 个兜底色
迁移范围	—	首页 / 论文库 / 发现 / 分析 / 图谱 / 写作 / 阅读 全部迁完
硬编码颜色残留	大量	六页清零（仅保留 ECharts 语义 alpha 值 3 处）

迁移过程中查出并修掉了两类真实缺陷，它们都不是「看起来不好看」，而是「实际坏了」：

- 首页引用了一个从未定义的变量 --line，浏览器静默回退成 border: none，分隔线整片消失。改为 --border-strong。

- 孤立文献标签的文字色对比度只有 2.31:1，低于 WCAG AA 要求的 4.5:1，看不清。改为 #5c6672 后达 5.30:1。

十二个角色色（基石 / 枢纽 / 桥接 / 基础 / 边缘 / 孤立 各三档前景背景）全部核对过对比度，最低 5.30:1，均达标。另外把原主题紫改成了蓝（#1a73e8，白底 4.51:1），因为紫色与引用高亮色撞色；但阅读页的参考文献回链色刻意保留紫色——它旁边的行内引用高亮已经是蓝色，两者必须能区分开。

有一个坑值得记：迁移后写了个检查脚本，扫出 settings.html 里用了 --text 这个变量，但设计系统里只有 --text-primary / --text-secondary / --text-muted。这种拼错不会有任何报错，只会让那段文字变成默认黑色。现在这条检查是常驻的：任何页面用到未定义的变量都会被列出来。

6.9 渲染性能与内存

赛题把「渲染性能与内存优化」单列 3 分，点名的三个场景是长 PDF 渲染、大图谱节点和上下文显存管理。以下是本项目的实际做法与在 2 核 4G 云服务器上的表现。

场景	问题	做法
541 页长 PDF	一次性渲染全部页面会吃掉数 GB 内存	pdf.js 按页 canvas 按需渲染，可视区外只保留文字层，离开视口即释放位图
36 节点 / 189 边图谱	全量重绘时 Layout 抖动	ECharts 增量更新节点，节点按社区分组渲染，避免每帧全量 force layout
全量建索引	峰值内存与耗时	分批读 PDF、流式写 chroma；实测 31623 个切块、1786 秒、峰值内存 876 MB
问答上下文	把全库片段塞进提示词会爆	三路召回只取前 50 个候选，重排后只取前 15 段，实际进提示词的最多 4 段

服务本身用 systemd 托管，配 MemoryMax=3G 作为硬上限；超出会被内核 OOM 杀掉，再由 Restart=always 在 5 秒内拉起。这套组合在上线前的验收里实测过：kill -9 之后 8 秒内恢复正常响应。另外配了每天 04:00 UTC（北京时间 12:00）的定时重启，把长沙时间里的内存缓慢增长截断在一天之内。

6.10 流式渲染与思考过程可视化

用户等一个需要检索多篇论文的回答时，最容易产生的疑虑是「它是不是卡住了」。本项目用 SSE（Server-Sent Events）把整个过程摊开给人看，共七类事件：

事件	内容	前端表现
roles	各角色执行记录与耗时（RoleRun）	时间线逐行出现，带角色名与毫秒数
tool_call	模型决定调用哪个工具、参数是什么	时间线插入一条工具调用卡片
tool_result	工具返回结果	卡片展开显示结果
evidence	检索到的证据片段	渲染成证据卡片，带篇名与页码
status	证据数 / 覆盖论文数 / 页码可定位性	回答上方的一行状态条，失败标红
answer_delta	答案正文的分增量	逐字流出，边生成边可读
done	结束标记	收起 loading 态

回答正文用 token 级流式推送，不是等全文生成完再一次性返回。这一点在演示时差别很明显：同样 8 秒的回答，逐字流出与空白 8 秒后整段蹦出来，用户的感受完全不同。

七、创新点

本章按赛题创新性评分的三个子项组织，避免与功能清单混在一起：b1 学术算法与图谱挖掘创新（10%）、b2 技术方案与 RAG 创新（8%）、b3 科研交互模式创新（7%）。每一项都写明「常规做法是什么 / 本项目怎么做 / 为什么」。

7.1 b1 学术算法与图谱挖掘创新（10%）

常规做法的三步与本项目的差异，集中在「宁缺毋滥」与「角色判定」两处。

环节	常规做法	本项目做法与理由
条目切分	按正则切年份或编号	多信号切分：标点、年份位置、作者块长度、缩进与续行标记综合判断。初版约 3300 条参考文献抽不出标题，全部卡在这一步
副本合并	按文件名或标题去重	标题 + arXiv 号 + 元数据交叉核验后合并成单一逻辑节点，但不删任何 PDF。直接建图会把被引数拆成多份，PageRank 失真
引用匹配	相似度超阈值就连边	三层匹配 arXiv > 完整标题 > 相似度，相似度 0.75~0.92 之间一律不出边，转人工核验。一条假边的破坏力远大于十条真边

图谱挖掘不满足于「画一张图看个大概」，而是给出可判断的结论：

- 角色判定：按入度 / 出度 / 中介中心性把节点分成基石、枢纽、桥接、基础、边缘、孤立六类。BERT 在本库中判为基石——被所有人引用、自己几乎不引别人。
- 三路前沿信号：新近且被引、跨社区桥接、尚未接入引用网，分别对应「热度」「交叉点」「空白区」三种科研选题来源。
- 自动综述：基于图结构与元数据生成研究现状概览，输入全部是可核验的结构化字段，不依赖大模型臆测。

值得注意的是 PageRank、中介中心性（Brandes）与社区发现全部用标准库自实现，刻意不依赖 networkx / scipy——依赖越少，评委与本机复现时踩坑越少。这条约束是有代价的（算法实现要自己写、自己要测），但换来的是 requirements.txt 更短、安装更稳。

7.2 b2 技术方案与 RAG 创新（8%）

创新点	常规做法	本项目做法
抗幻觉	生成后校验，不合格重写	约束前移到生成结构：文献池由代码提供、引用只能写 key、参考文献由代码从池里取。模型没有写完整条目的权限
多路召回	单路向量检索，或简单分数相加	三路召回 + RRF 只用名次融合。三路原始分数量纲完全不同（余弦 0~1、BM25 无上界、哈希内积可正可负），直接相加等于让量纲大的通道主导结果
校验环节	检查失败就报错返回	Verifier 不合格会打回检索层换策略重试：先回填页码、再换重排信号、再放宽条数。这是整条链路唯一真正能改变流程的一环
版本差异	只提示「库中已有更新版本」	实测 v1 与 v7 相似度 0.873，提示已有更早版本但不拦着上传——科研里对比不同版本是真实需求

另外两个来自真实调试、不是设计推演得来的一致性决策：交叉编码器输出的是 logits（可正可负，只在批内可比），前端曾把它标成「融合分」导致出现负数分数，现按 score_type 区分显示；引用降权系数 0.6 放在重排之后而不是之前——如果先降权，参考文献条目会挤掉正文段落，而正文才是答案的主要来源。

7.3 b3 科研交互模式创新（7%）

交互点	常规做法	本项目做法
阅读页	把 PDF 塞进 iframe 或简单预览	重新渲染：底层页面图像 + 上层透明文字层，文字可选、可定位、可按章节跳转
引用跳转	正文 [11] 单向跳到文末	双向无损往返：正文跳文末第 11 条，那边点返回跳回原处，记住你从哪来的
中英对比	两栏各自滚动	两侧拖动自动同步滚动。硬难点是 pdf.js 文本项之间没有分隔符，句边界定位失败，需要按字符宽度重算
翻译	整篇翻译或划词翻译二选一	三种模式并存：整篇左右对照、划词、悬停（鼠标停 450ms 旁边弹中文）。代码与公式保留原文不翻，这是学术翻译的硬要求

证据溯源	只给一段文字说「来自第 7 页」	每条结论下挂证据卡片，点一下跳到 PDF 那一页并高亮原文，可以自己核对

7.4 其他值得记录的工程细节

这些细节不属于三大创新点，但都是真实调试出来的，不是设计推演的产物：Graphviz 中文豆腐块（默认字体不含 CJK 字形，需渲染前注入）、mermaid-cli 在 Windows 上的 shutil.which 陷阱（会优先命中 npm 的无扩展名脚本，报「不是有效的 Win32 应用程序」）、对数轴阈值取 100 倍（参数量跨度 1600 倍用对数轴，准确率 86~93% 不跨数量级反而不能对数）、雷达图逐列归一（各维量纲不同会互相吃掉）、以及 caption 生成只给统计摘要不给图（给图模型会描述出图上并不存在的趋势）。这些已写入 README 的技术决策记录。

八、AI 协同贡献与 Prompt 调优策略

赛题规则 3.4 明确要求：允许并鼓励使用 AI 辅助编程工具，但参赛者须理解并掌控项目的核心架构逻辑，并在技术文档中清晰说明 AI 的协同贡献与 Prompt 调优策略。本章如实交代这件事，包括 AI 帮上了什么、以及哪里必须由人接手。

8.1 AI 参与的部分与人工掌控的部分

环节	AI 参与方式	人工掌控点
架构与模块划分	按人工给出的模块边界补写实现	分层、单进程形态、四角色命名、哪些功能砍掉，全部由人决定
提示词初稿	生成第一版提示词	角色约束、拒答条件、输出格式、few-shot 示例逐条审改
缺陷定位	辅助阅读堆栈、提出假设	所有根因判断都回到代码与实测数据验证后才采纳
测试补齐	按人工描述的用例意图写测试	测试该断言什么、哪些不确定逻辑不该断言，由人定
文档与讲稿	按提纲成稿	所有数字必须由脚本实测得出，不允许 AI 直接写数

一条硬纪律贯穿全程：文档里出现的每一个数字，都必须有一条能复现它的命令。本项目曾出现过「测试 364 条」「接口 33 个」这类沿用了很久的旧数字，就是靠逐个跑命令才发现的。现在这些数字由 scripts/build_tech_doc.py 在生成文档时现场统计，比如测试数直接调 pytest --co 取真实收集数（当前 444 条），路由数扫 app/main.py 的装饰器（当前 44 个）。

8.2 Prompt 调优策略

本项目的提示词不是「一段话反复调」，而是按角色拆开，每个角色解决一类确定的问题。调优主要围绕四条策略：

策略	具体做法	解决的什么病
结构约束优先于措辞约束	不写「请不要编造」，而是让它没有编造的入口：参考文献由代码生成	措辞约束在长上下文里会被稀释，结构约束不会

拒答条件显式写死	证据不足时必须输出「资料不足」，并给出缺什么	模型天然倾向于给一个像样的答案，哪怕没有依据
输出格式由代码兜底	要求标 [C1] 这类 key，生成后校验池外 key 即判失败	模型偶尔会自创编号，只靠提示词拦不住
上下文只给必要的	重排后只把最相关的若干片段交给模型	塞满上下文既贵又让模型注意力涣散

另外记录一次真实的提示词事故：图表 caption 生成最初把统计摘要和图一起给模型，结果模型描述出了图上并不存在的趋势。修法是只给统计摘要、不给图——这个改动把「模型看图猜」变成「模型复述数据」，从根上消除了那类幻觉。

九、验收与复现

9.1 快速启动

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 配置密钥（复制模板后填入自己的 Key）
cp .env.example .env

# 3. 启动服务
uvicorn app.main:app --host 127.0.0.1 --port 8000

# 4. 浏览器打开
http://127.0.0.1:8000/ui/index.html
```

9.2 自检脚本

脚本	作用
scripts/check_all.py	环境、索引、模型、Git、依赖一键体检
scripts/check_deps.py	依赖清单与代码 import 一致性检查
scripts/build_structure.py	重建细粒度解析结果
scripts/build_graph.py	重建引用图谱
scripts/build_index.py	重建向量索引
scripts/verify_official5.py	官方 1.1~1.4 条目验收脚本

9.3 质量门禁

```
ruff check .                # All checks passed!
pytest tests -q             # 444 passed
python scripts/check_deps.py # ✓ 依赖清单覆盖全部 import
```

以上三项已配置在 .github/workflows/ci.yml 中，每次推送自动执行。

十、已知局限与后续工作

局限	现状	后续方向
低置信度匹配需人工核验	48 条落在阈值之间，未入图	引入引用上下文语义判断，降低误判
多智能体未做模型级协作	四阶段显式命名，非独立模型对话	在延迟允许时引入 Planner 独立推理
外部库依赖网络	arXiv / Crossref 查询需联网	增加本地元数据缓存与降级路径
容器内渲染器不全	只装了 Graphviz	按需提供 Mermaid/TikZ 的镜像变体
重排模型需手工放置	data/models/ 不入库，缺失时降级	提供下载脚本与校验和

上述局限均已记录在仓库 README 与 docs/ 下的交接文档中，没有以「已实现」的方式掩盖。其中第一条与第五条是后续最优先处理的两项。